

Python Programming Assignment

Comprehensive Problem Set: Basics, Data Structures, Functions & OOP Foundations

Course Component

Foundations to OOP

Total Questions

20 Problems

Target Outcome

NumPy & Pandas Prep

Objective: This assignment evaluates core Python proficiency before transitioning into Data Analysis libraries (NumPy & Pandas). Focus on clean logic, proper error handling, and object-oriented structure.

MODULE 1: BASICS & CONTROL FLOW

Level 1 – Beginner

Q1. Temperature Converter & Weather Advice

Write a program that converts temperatures between Celsius and Fahrenheit based on user input. Prompt for the scale (`C` or `F`) and the numerical degree.

- Formula: $F = (C \times 9/5) + 32$ | $C = (F - 32) \times 5/9$
- If converted Celsius value is < 10 , print "Cold"; if between 10 and 28, print "Mild"; if > 28 , print "Hot".

Q2. Number Classifier & Stat Collector

Write a function `analyze_numbers(numbers)` that accepts a list of integers and returns a dictionary containing summary facts without using built-in functions (`sum` , `min` , `max` , `len`):

- Keys required: `total_count` , `sum_values` , `even_count` , `odd_count` , `min_val` , and `max_val` .

Q3. Pattern Printing Engine

Write a function `print_diamond(n)` that takes an odd integer n and prints a symmetric diamond of asterisks (`*`). If an even integer is provided, raise a `ValueError` with a custom message.

Q4. Prime Range Finder

Implement `find_primes_in_range(start, end)` using nested loops. Return a list of all prime numbers between `start` and `end` (inclusive). Optimize by checking divisibility only up to $\sqrt{n}$.

Q5. Mathematical Sequences

Generate the first N terms of the Fibonacci sequence using an iterative approach, storing the results in a list. Ensure your function handles inputs of $N = 0$ and $N = 1$ gracefully.

MODULE 2: DATA STRUCTURES & STRING MANIPULATION

Level 2 – Intermediate

Q6. Text Cleaning & Word Frequency

Write a function `clean_and_count(text)` that takes a string paragraph, removes punctuation characters, converts text to lowercase, and returns a dictionary with word frequencies sorted by count descending.

Q7. List Flattening & De-duplication

Write a function `flatten_and_unique(nested_list)` that takes a multi-level nested list (e.g., `[1, [2, 3, [4, 1]], 5]`) and flattens it into a 1D list with unique elements preserved in order of appearance.

Q8. Matrix Transpose & Addition (Pre-NumPy Prep)

Implement 2D list manipulation functions manually using list comprehensions:

- `transpose_matrix(matrix)` : Flips rows and columns.
- `add_matrices(mat_a, mat_b)` : Adds two matrices of identical shape element-wise.

Q9. Dictionary Merging & Aggregation

Given a list of store sales dictionaries, e.g., `[{"apple": 10, "banana": 5}, {"apple": 15, "orange": 8}]`, write a function that aggregates total sales per item across all stores into a single summary dictionary.

Q10. Two-Sum Problem

Given a list of integers and a target number, write a function `two_sum(nums, target)` that returns the indices of the two numbers that add up to target. Optimize the lookup time to $O(n)$ using a dictionary.

MODULE 3: FUNCTIONS & MODULAR PROGRAMMING

Level 2 – Intermediate

Q11. Functional Grade Filter

Given a list of student dictionary records containing names and list of marks, use functional utilities (`map`, `filter`, `lambda`) to compute average scores and return names of students scoring above 75%.

Q12. Flexible Argument Aggregator

Create a function `calculate_stats(*args, **kwargs)` that accepts arbitrary numerical arguments alongside operational keyword flags like `scale=10` or `operation="mean"` to dynamically output calculated metrics.

Q13. Custom File Parser & Error Logger

Write a script that reads a text file containing numerical data lines. Handle `FileNotFoundError` if the path is invalid and `ValueError` for malformed rows. Log errors to a separate `error.log` file and return the mean of valid rows.

Q14. Recursive Array Search

Write a recursive function `recursive_search(data, target)` that searches for an element inside a nested list of arbitrary depth and returns `True` if found, otherwise `False`.

MODULE 4: OBJECT-ORIENTED PROGRAMMING (OOP)

Level 3 – Advanced

Q15. Bank Account Hierarchy

Implement a class structure demonstrating Encapsulation and Inheritance:

- `BankAccount` : Base class with private attribute `__balance`, methods `deposit()`, `withdraw()`, and getter `get_balance()`. Raise `InsufficientFundsError` if withdrawal exceeds balance.
- `SavingsAccount` : Derived class with `interest_rate` attribute and a method `apply_interest()`.

Q16. E-Commerce Shopping Cart System

Build an interactive shopping cart system using OOP principles:

- `Item` class: Stores `name`, `price`, and `quantity`. Includes a method `get_total_price()`.
- `ShoppingCart` class: Contains a collection of `Item` objects. Implements methods to `add_item()`, `remove_item()`, and `calculate_grand_total()` with tax application.

Q17. Shape Polymorphism & Area Calculator

Create an abstract interface `Shape` with an abstract method `area()`. Implement concrete subclasses `Circle`, `Rectangle`, and `Triangle`. Write a function `total_area(shapes)` that accepts a list of shapes and computes total combined area using polymorphism.

Q18. Custom Vector Class (Operator Overloading)

Create a class `Vector2D` representing a 2D mathematical vector with attributes `x` and `y`. Overload standard Python magic methods to enable vector operations directly:

- `__add__`: Vector addition ($v1 + v2$).
- `__sub__`: Vector subtraction ($v1 - v2$).
- `__mul__`: Scalar multiplication or dot product.
- `__str__`: User-friendly string display format `"Vector(x, y)"`.

Q19. Dataset Row & Normalizer (Pandas Prep)

Design a `DataPoint` class to mimic table row operations in data analysis:

- **Attributes:** `id`, `feature_vector` (list of numerical values), and `label`.
- **Method `normalize()`**: Performs min-max normalization on `feature_vector` transforming values into a scale between 0 and 1:

$$x_{norm} = (x - x_{min}) / (x_{max} - x_{min})$$

- **Method `impute_missing(fill_value)`**: Replaces any `None` entries in the feature list with `fill_value`.

Q20. Custom DataFrame Series Structure (Mini-Pandas)

Implement a class `SimpleSeries` that wraps a list of data alongside custom string indices:

- **Constructor:** Accepts a list of data values and an optional list of index labels.
- **Methods:**
 - `mean()` and `std()`: Compute arithmetic mean and standard deviation.
 - `filter_above(threshold)`: Returns a new `SimpleSeries` filtered by condition.
 - `head(n)`: Returns the first `n` elements with their corresponding index labels.

Next Steps: Completing these 20 problems ensures a solid foundation in data structures, control logic, clean functional practices, and OOP design needed to excel with NumPy arrays and Pandas DataFrames.